Laporan Tugas Kecil I
Mata Kuliah IF 2211 Strategi Algoritma

Disusun oleh:

13524010-Audric Yusuf Maynard Simatupang
13524010@std.stei.itb.ac.id
13524052-Raynard Fausta
13524052@std.stei.itb.ac.id

Program Studi
Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Bab I: Pendahuluan

1.1. Latar Belakang

Pengembangan teknologi untuk memodelkan bentuk 3D sudah semakin pesat pada saat ini. Namun, objek yang awalnya direpresentasikan dalam bentuk permukaan yang kontinu(mesh) perlu diubah menjadi representasi yang lebih terstruktur sehingga dapat lebih mudah diproses oleh komputer. Salah satu representasi yang sering digunakan adalah voxel(elemen volumetrik berbentuk kubus kecil yang menyusun sebuah objek 3 dimensi). Proses tersebut dinamakan *Voxelization*.

Akan tetapi, proses *voxelization* memiliki permasalahan dalam hal efisiensi penyimpanan dan pengelolaan data. Semakin tinggi tingkat detail objek yang ingin direpresentasikan, semakin besar juga jumlah voxel yang harus disimpan dan diproses sehingga dibutuhkan struktur data yang mampu mengatur pembagian ruang secara efisien.

Octree menjadi struktur data yang paling efektif untuk digunakan karena dapat membagi ruang tiga dimensi menjadi delapan bagian pada setiap percabangan(partisi). Struktur data tersebut sangat adaptif sehingga area yang membutuhkan detail lebih tinggi dapat dibagi lebih lanjut dan mengurangi penyimpanan pada area yang kosong atau homogen.

Berdasarkan hal-hal tersebut, penggunaan *Octree* dalam proses *voxelization* menjadi solusi yang relevan untuk menghasilkan representasi tiga dimensi yang lebih efisien dan terstruktur.

1.2. Tujuan

Laporan ini bertujuan untuk membuktikan bahwa algoritma *Divide and Conquer* dapat dimanfaatkan untuk melakukan *Voxelization* Objek 3D menggunakan *Octree*.

Bab II: Landasan Teori

2.1. Algoritma Divide and Conquer

Algoritma *Divide and Conquer* merupakan algoritma yang terdiri atas 3 tahapan umum, yaitu *divide*, *conquer,* dan *combine.* Pada tahap *divide,* algoritma akan memecah persoalan menjadi subpersoalan yang lebih kecil untuk diselesaikan. Kemudian pada tahap *conquer*, subpersoalan yang telah dibagi akan diselesaikan. Pada tahap *combine*, semua solusi yang telah dihasilkan akan digabungkan sehingga menghasilkan sebuah solusi akhir untuk persoalan awal.

2.2. Voxelization

Proses mengubah representasi objek menjadi *voxel*.

2.3. Voxel

Elemen volumetrik yang berbentuk kubus kecil dan menyusun sebuah objek 3 dimensi

2.4. Octree

Struktur data berbentuk pohon hierarkis yang digunakan untuk merepresentasikan dan membagi ruang tiga dimensi secara rekursif. Setiap simpul internal memiliki delapan anak yang mewakili delapan subruang hasil pembagian ruang 3 dimensi menjadi delapan bagian yang sama besar.

Bab III: Metode Penyelesaian

3.1. Metode Divide and Conquer

```java
package backend.logic;

import backend.data.*;
import java.util.List;
import java.util.concurrent.RecursiveTask;
import java.util.ArrayList;

public class RecurseOctree extends RecursiveTask<OctreeResult> {
    OctreeTask octreeTask;
    Model model;

    public RecurseOctree(OctreeTask octreeTask, Model model) {
        this.octreeTask = octreeTask;
        this.model = model;
    }

    @Override
    protected OctreeResult compute() {
        OctreeResult octreeResult = new
OctreeResult(octreeTask.getMaxDepth());
        octreeResult.incrementCreated(octreeTask.getDepth());

        if (!CalculatorUtil.isIntersectModel(model,
octreeTask.getCube())) {
            octreeResult.incrementSkipped(octreeTask.getDepth());
            return octreeResult;
        }

        if (octreeTask.getDepth() == octreeTask.getMaxDepth()) {
            octreeResult.addVoxel(octreeTask.getCube());
            return octreeResult;
        }

        Cube[] childCubes = octreeTask.getCube().split();
        List<RecurseOctree> tasks = new ArrayList<>();

        for (Cube cube : childCubes) {
```

```
            tasks.add(new RecurseOctree(new OctreeTask(cube,
octreeTask.getDepth() + 1, octreeTask.getMaxDepth(), model), model));
        }

        for (int i = 0 ; i < 7 ; i++) {
            tasks.get(i).fork();
        }

        OctreeResult totalResult = tasks.get(7).compute();

        for (int i = 0 ; i < 7 ; i++) {
            totalResult.mergeResult(tasks.get(i).join());
        }

        octreeResult.mergeResult(totalResult);
        return octreeResult;



    }
}
```

Pada implementasi *voxelization*, metode *divide and conquer* diterapkan melalui struktur data *Octree* untuk membagi ruang 3 dimensi secara bertahap dan adaptif. Proses dimulai dengan pembentukan *root cube*(kubus awal yang membungkus seluruh model 3 dimensi berdasarkan nilai koordinat minimum dan maksimum dari seluruh *vertex*). Setelah itu, algoritma memeriksa apakah kubus beririsan dengan geometri model yang dilakukan dengan membandingkan setiap *face* segitiga pada model terhadap kubus yang sedang diproses. Jika suatu kubus tidak memiliki irisan dengan model maka proses pada cabang tersebut langsung dihentikan sehingga lebih efisien dalam proses *divide and conquer* karena ruang kosong tidak perlu dipecah lebih lanjut.

Tahap *divide* dilakukan ketika sebuah kubus masih beririsan dengan model, tetapi tingkat kedalamannya belum mencapai batas maksimum. Pada kondisi tersebut, kubus dibagi menjadi delapan subkubus yang berukuran lebih kecil melalui operasi split() yang disesuaikan dengan karakteristik utama *Octree*. Pembagian menjadi tahap yang paling krusial karena harus ditentukan apakah kubus tersebut harus dibuang, dijadikan *voxel*, atau dibagi menjadi 8 kubus yang lebih kecil lagi. Hal ini bertujuan agar algoritma hanya meningkatkan ketelitian representasi pada area yang berkaitan dengan bentuk model dan tidak membuang sumber daya komputasi pada seluruh ruang secara merata.

Tahap *conquer* dilakukan dengan menyelesaikan setiap submasalah tersebut menggunakan prosedur yang sama secara rekursif. Setiap subkubus diperiksa apakah beririsan dengan model, apakah harus dihentikan, atau perlu dibagi lagi. Jika suatu subkubus tidak beririsan dengan model, maka cabang tersebut dipangkas. Sebaliknya, jika subkubus masih beririsan dan telah mencapai maxDepth, maka

subkubus tersebut diterima sebagai *voxel* akhir dan disimpan sebagai bagian dari hasil *voxelization*. Hal ini bertujuan agar rekursi tidak berlangsung tanpa batas dan dapat dihentikan berdasarkan kondisi geometris dan kedalaman pohon.

Tahap *combine* terjadi setelah seluruh delapan subkubus hasil pembagian selesai diproses. Hasil dari masing-masing anak, baik berupa voxel yang terbentuk maupun data statistik, seperti jumlah node yang dibuat dan jumlah node yang dilewati, digabung kembali ke node induknya melalui proses mergeResult(). Dengan demikian, node induk tidak hanya berfungsi sebagai pemecah masalah, tetapi juga penggabung hasil dari seluruh subtree di bawahnya. Pada akhirnya, hasil dari seluruh rekursi tersebut membentuk representasi *voxel* untuk keseluruhan model.

Bab IV: Hasil dan Pembahasan

4.1. Analisis Kompleksitas

Pada setiap iterasi, apabila kubus masih beririsan dengan model dan belum mencapai *max depth* maka akan dibagi menjadi 8 subkubus. Masing-masing subkubus akan diselesaikan secara independen dan pada akhirnya digabungkan. Pada setiap rekursi, untuk mendapatkan model dengan detail terbaik dilakukan pemeriksaan apakah *face* objek telah beririsan dengan kubus(apakah di *max depth* atau bukan) atau tidak sama sekali.

Oleh karena itu, kompleksitas algoritmanya dapat dirumuskan sebagai berikut:

$$T(n) = \begin{cases} m & n = 1 \\ 8T(n/8) + m & n > 1 \end{cases}$$

n= volume subruang, m= *face*

Karena jumlah *face* tidak dipengaruhi oleh jumlah volume subruang maka dapat dianggap sebagai konstanta sehingga dapat dituliskan ulang menjadi:

$$T(n) = \begin{cases} c & n = 1 \\ 8T(n/8) + cn^0 & n > 1 \end{cases}$$

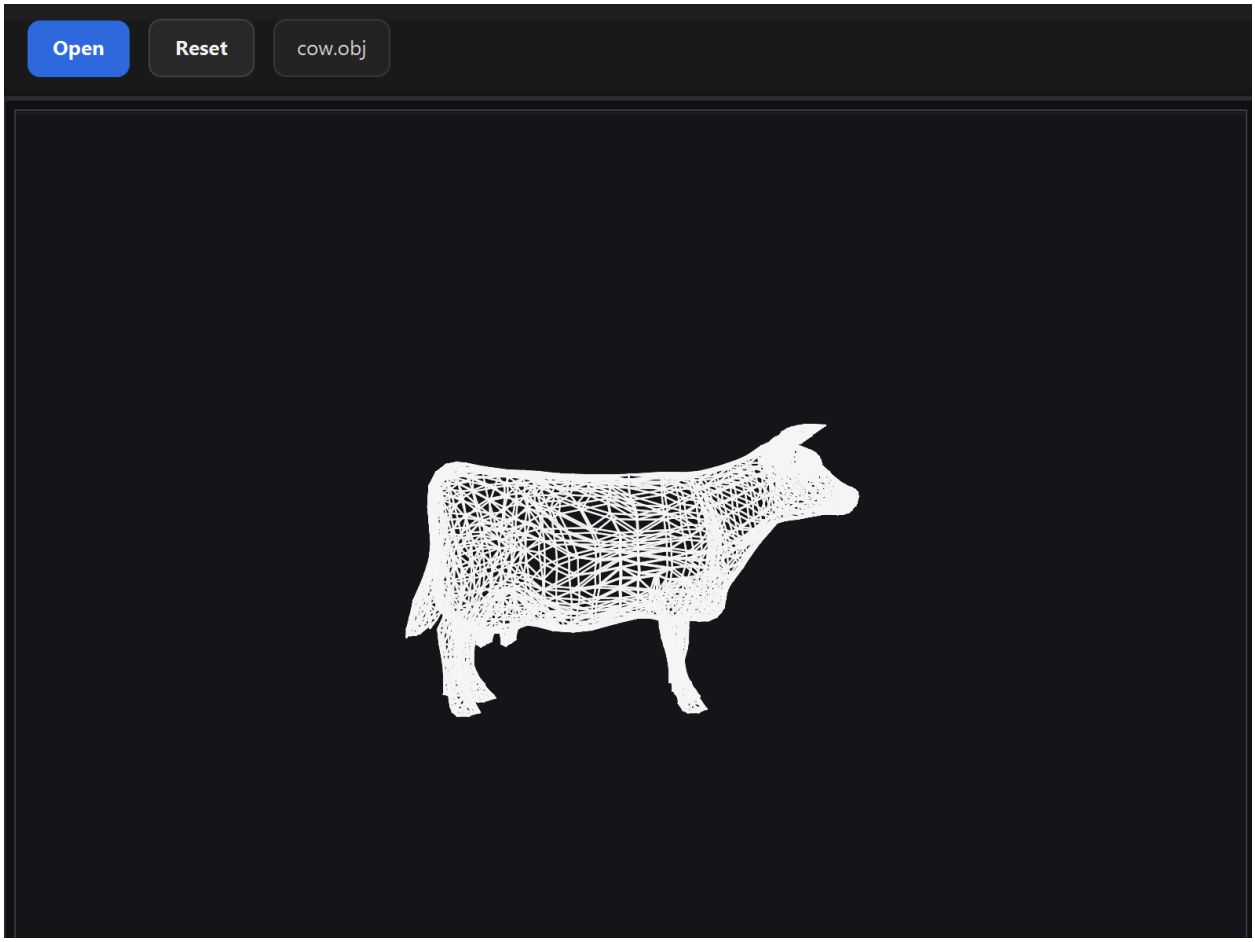
Nilai a=8, b=8, c=c, dan d=0. Nilai a>b^d, yaitu 8>8^0 = 8>1. Oleh karena itu, menurut teorema master, kompleksitas big O asimtotiknya adalah O($n^{log_b a}$)=O($n^{log_8 8}$)=O(n^1)=O(n).

4.2. Hasil Percobaan

Berikut adalah hasil percobaan testing input :

1. cow.obj

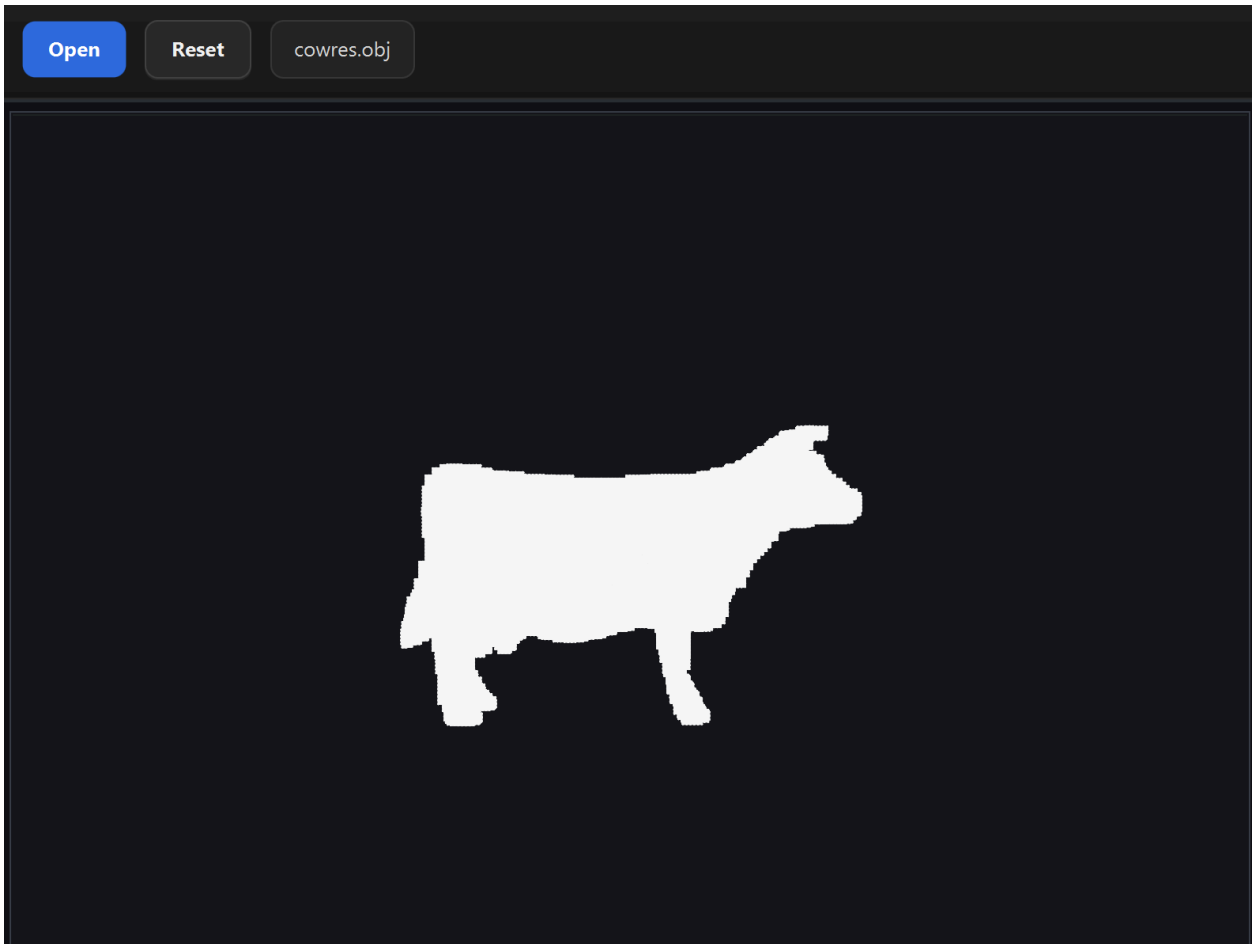
   Input : cow.obj



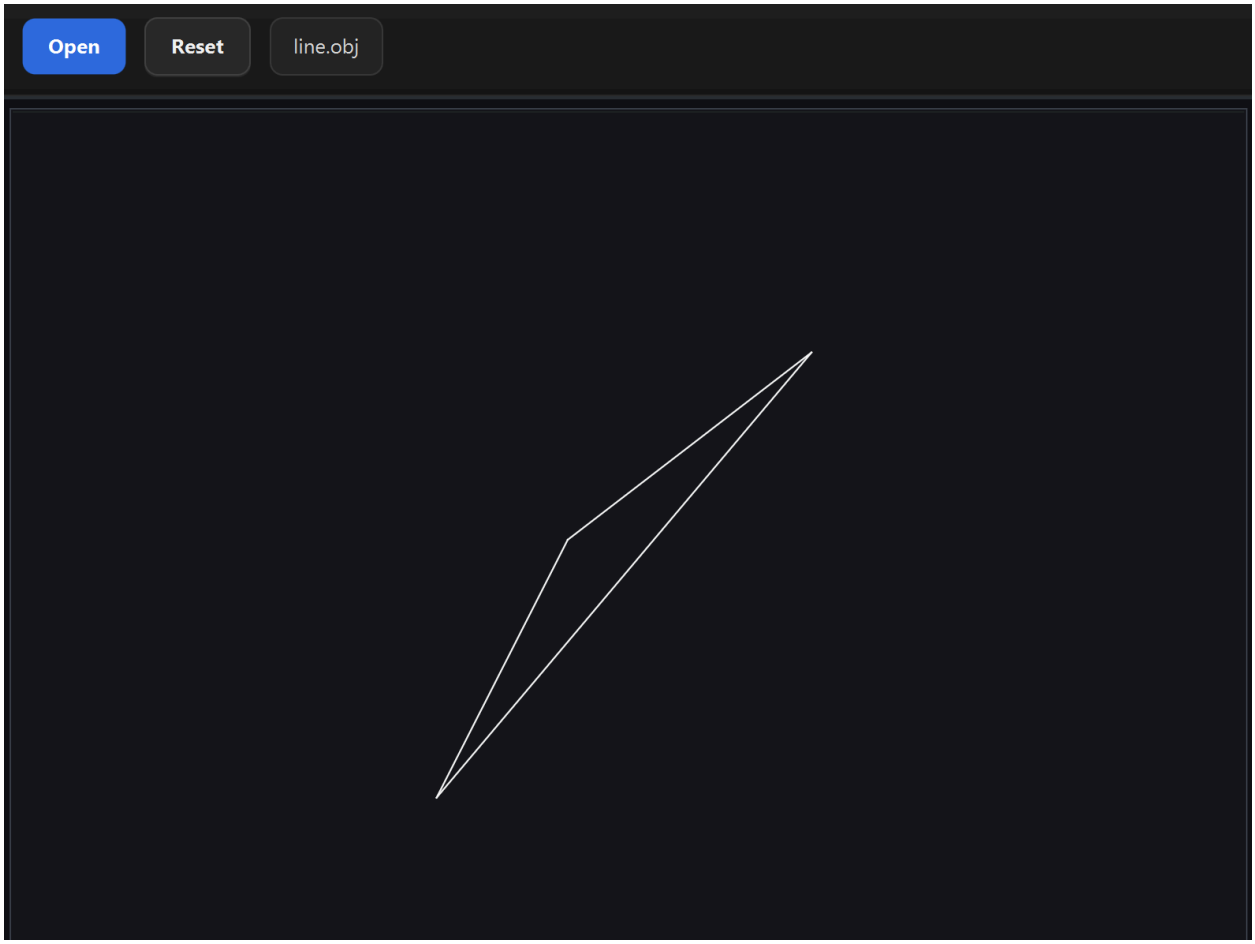
Output : [cowres.obj]

```
Masukkan file path :
cow.obj
Masukkan max depth :
5
Masukkan nama file output (.obj) :
cowres.obj

Banyak voxel yang terbentuk: 1469
Banyak vertex yang terbentuk: 11752
Banyak faces yang terbentuk: 17628
Statistik node octree yang terbentuk:
0 : 1
1 : 8
2 : 32
3 : 160
4 : 720
5 : 2880
Statistik node yang tidak perlu ditelusuri:
0 : 0
1 : 4
2 : 12
3 : 70
4 : 360
5 : 1411
Kedalaman octree: 5
Lama waktu program berjalan: 147.5648 ms
Path file output: cowres.obj
```



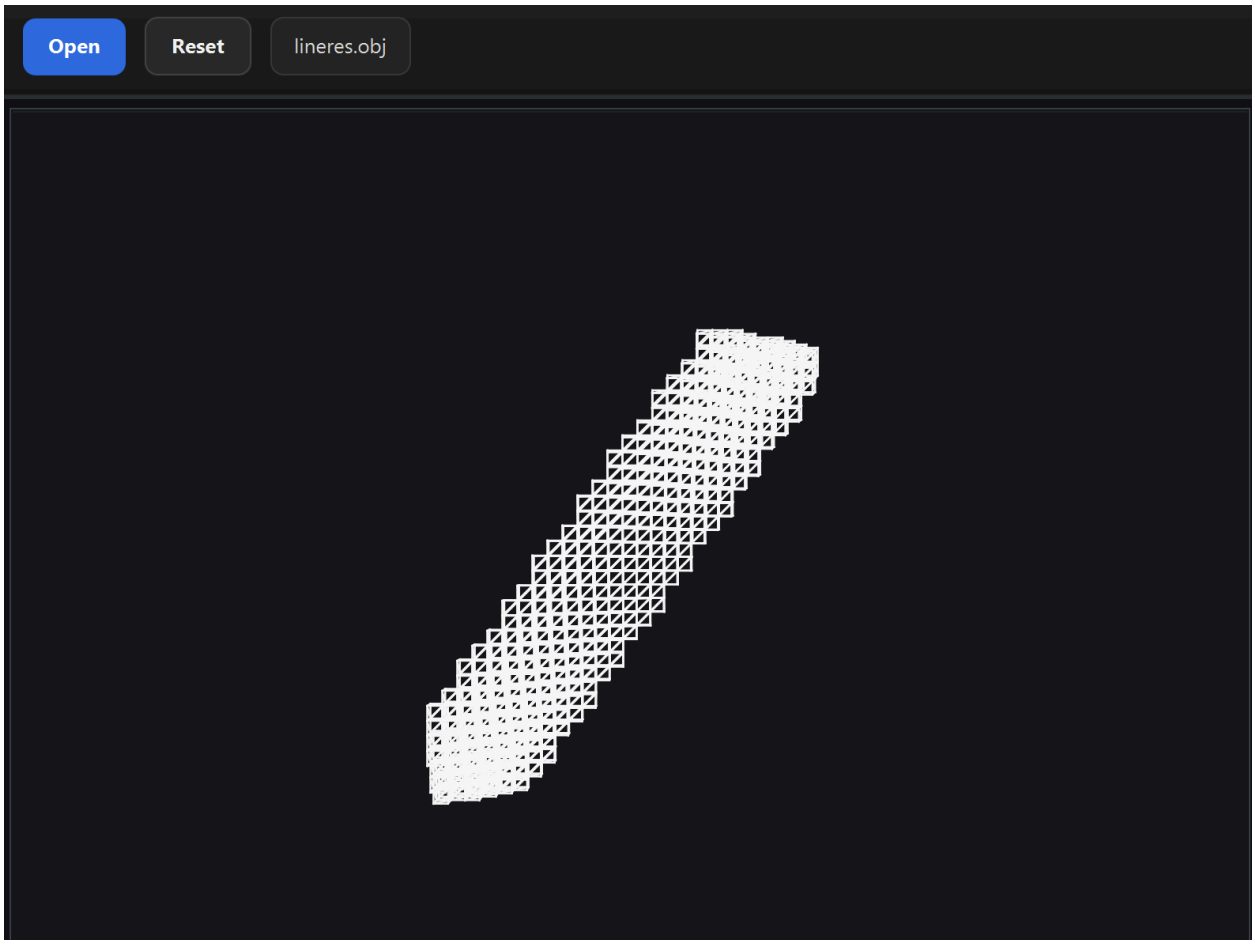
2. line.obj
   Input : line.obj



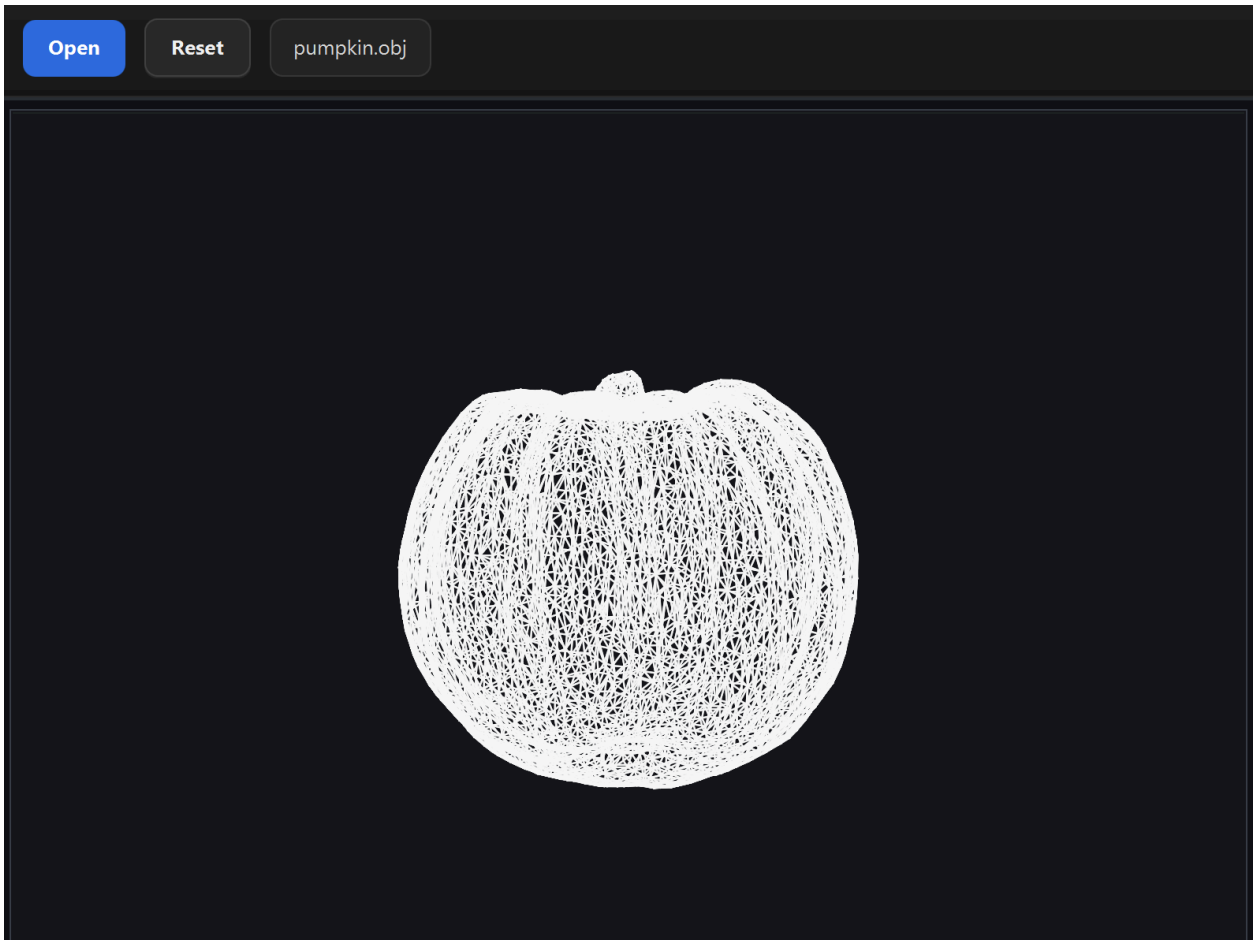
Output : [lineres.obj]

```
Masukkan file path :
line.obj
Masukkan max depth :
5
Masukkan nama file output (.obj) :
lineres.obj

Banyak voxel yang terbentuk: 634
Banyak vertex yang terbentuk: 5072
Banyak faces yang terbentuk: 7608
Statistik node octree yang terbentuk:
0 : 1
1 : 8
2 : 32
3 : 88
4 : 352
5 : 1448
Statistik node yang tidak perlu ditelusuri:
0 : 0
1 : 4
2 : 21
3 : 44
4 : 171
5 : 814
Kedalaman octree: 5
Lama waktu program berjalan: 30.9292 ms
Path file output: lineres.obj
```



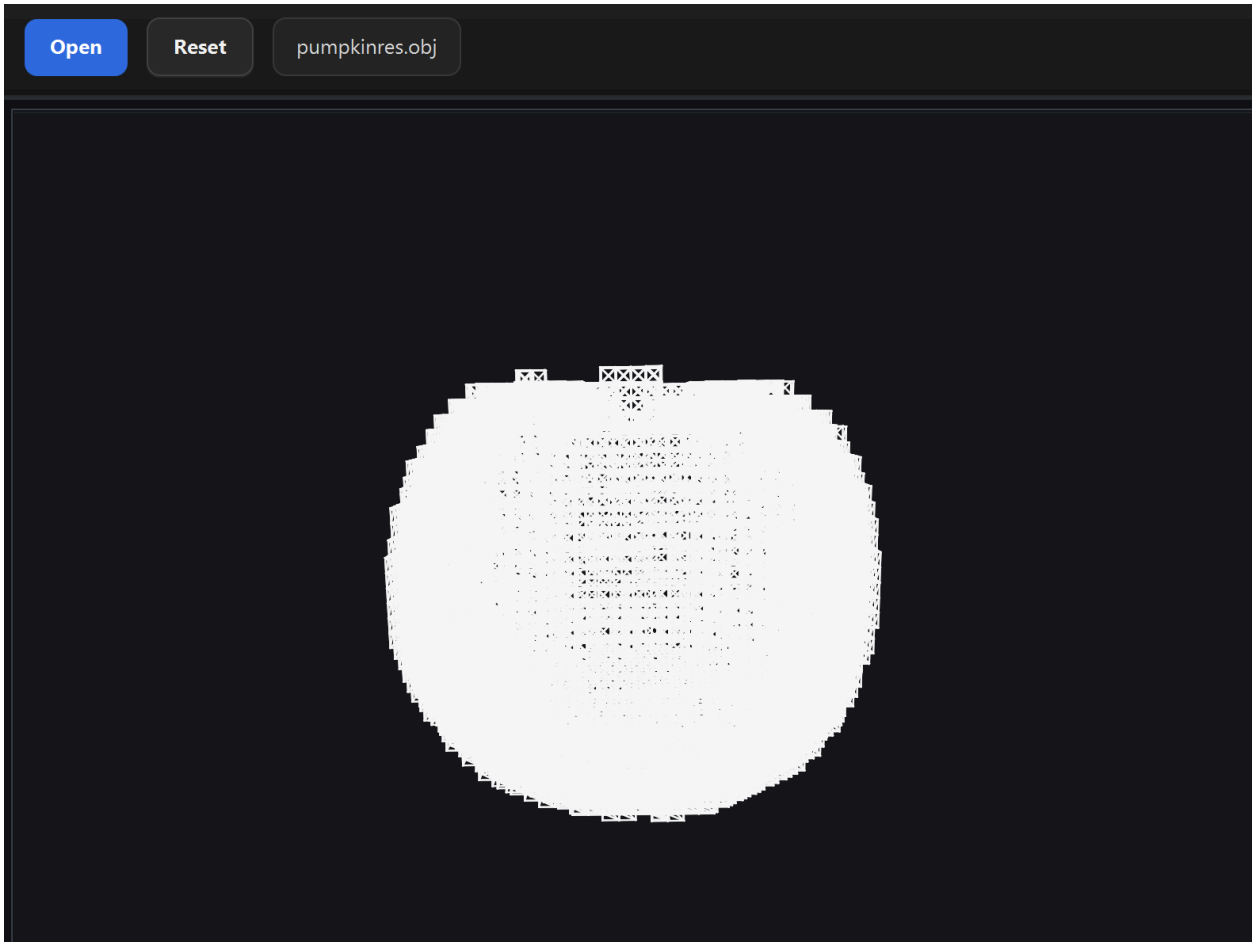
3. pumpkin.obj
   Input : [pumpkin.obj]



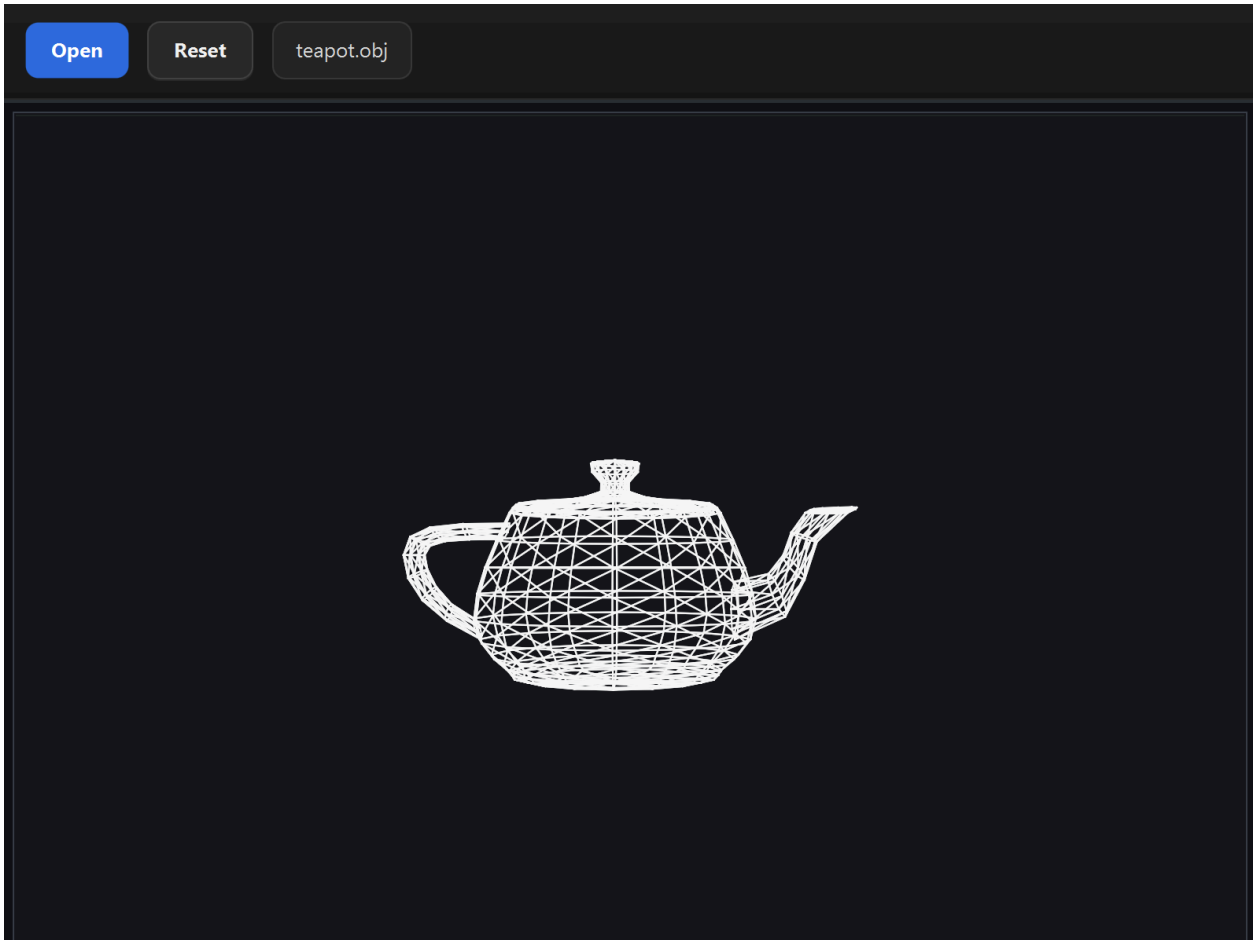
Output : [pumpkinres.obj]

```
Masukkan file path :
pumpkin.obj
Masukkan max depth :
5
Masukkan nama file output (.obj) :
pumpkinres.obj

Banyak voxel yang terbentuk: 4834
Banyak vertex yang terbentuk: 38672
Banyak faces yang terbentuk: 58008
Statistik node octree yang terbentuk:
0 : 1
1 : 8
2 : 64
3 : 448
4 : 2072
5 : 9288
Statistik node yang tidak perlu ditelusuri:
0 : 0
1 : 0
2 : 8
3 : 189
4 : 911
5 : 4454
Kedalaman octree: 5
Lama waktu program berjalan: 171.567 ms
Path file output: pumpkinres.obj
```



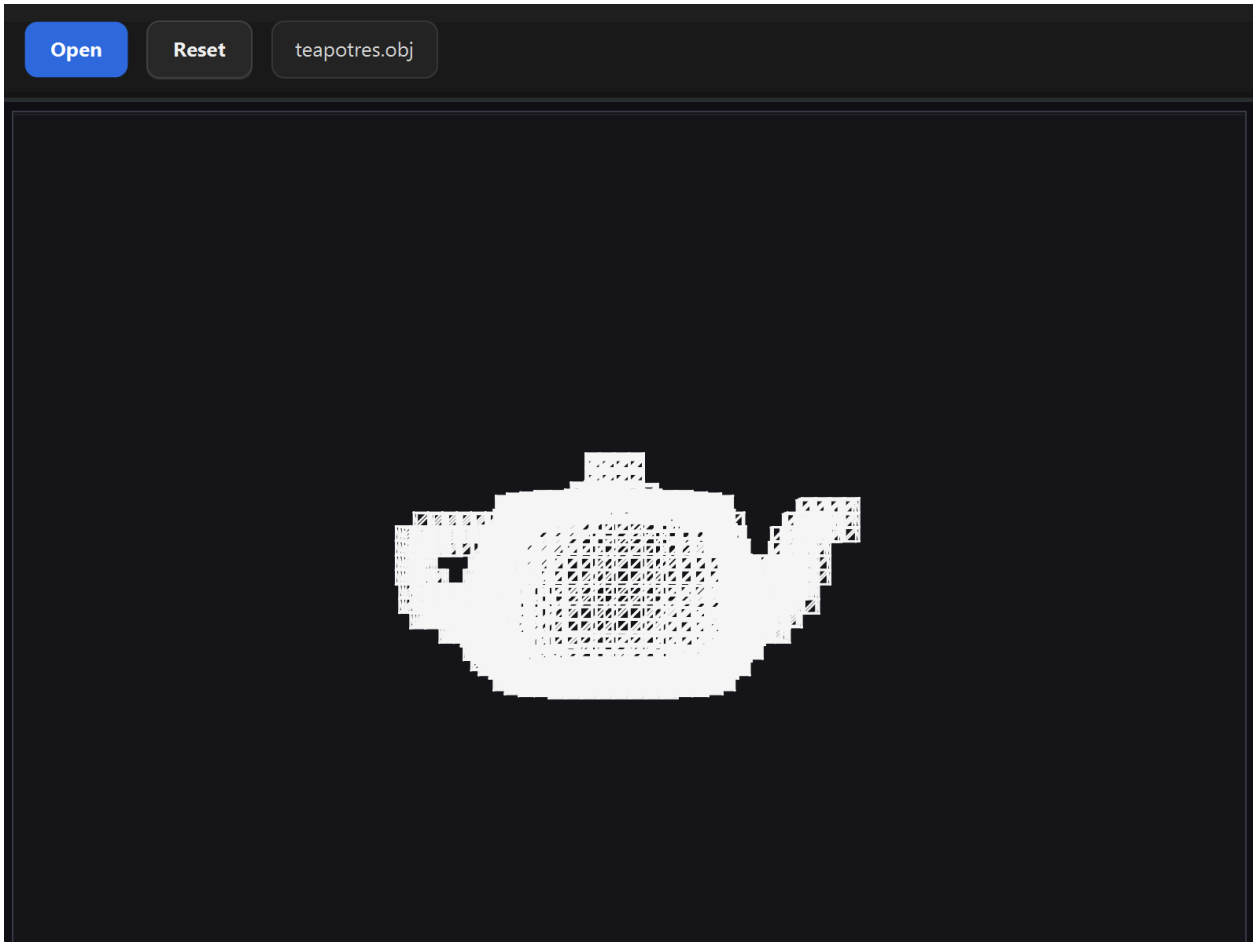
4. teapot.obj
   Input : [teapot.obj]



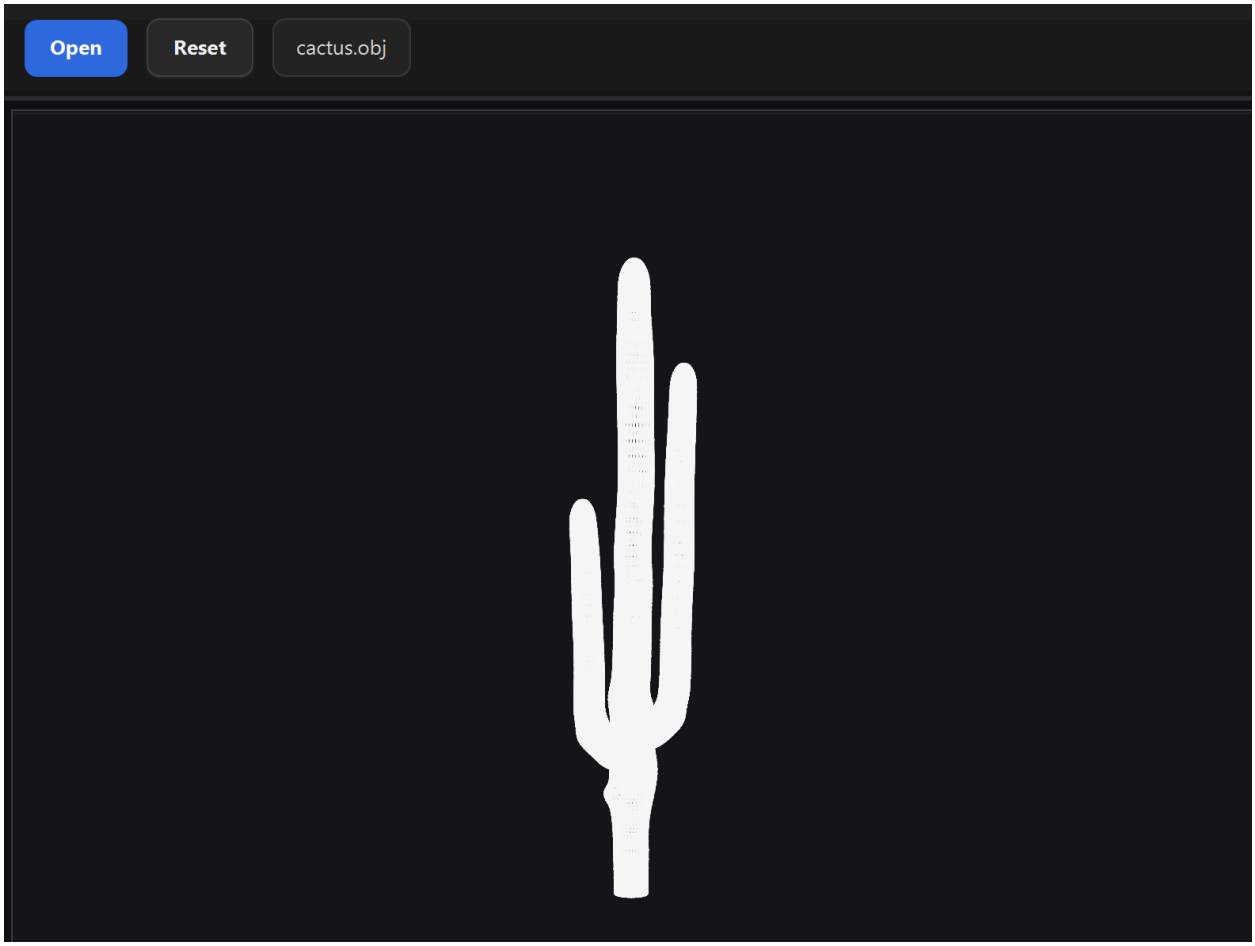
Output : [teapotres.obj]

```
Masukkan file path :
teapot.obj
Masukkan max depth :
5
Masukkan nama file output (.obj) :
teapotres.obj

Banyak voxel yang terbentuk: 1870
Banyak vertex yang terbentuk: 14960
Banyak faces yang terbentuk: 22440
Statistik node octree yang terbentuk:
0 : 1
1 : 8
2 : 32
3 : 168
4 : 800
5 : 3568
Statistik node yang tidak perlu ditelusuri:
0 : 0
1 : 4
2 : 11
3 : 68
4 : 354
5 : 1698
Kedalaman octree: 5
Lama waktu program berjalan: 87.3846 ms
Path file output: teapotres.obj
```



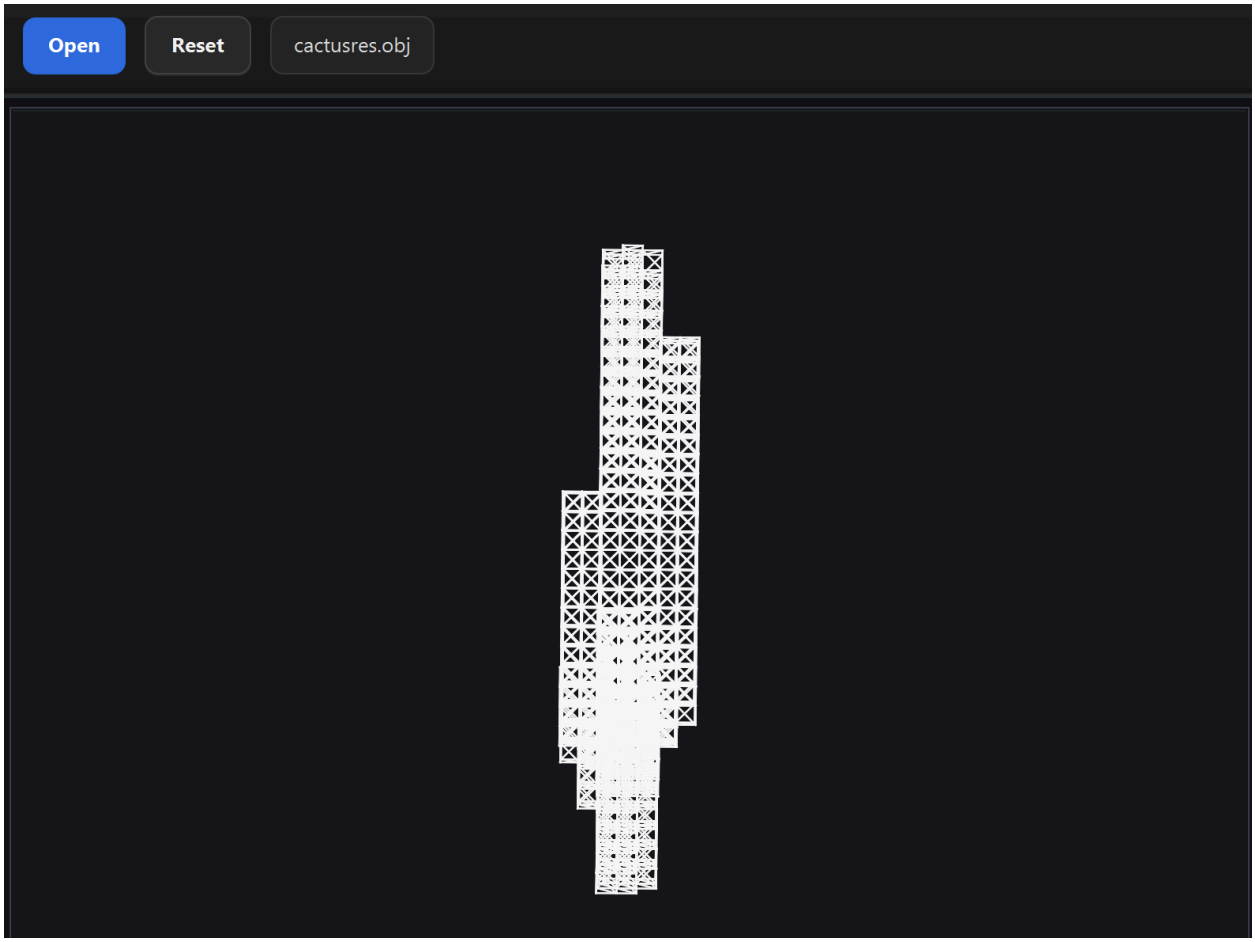
5. cactus.obj
   Input : [cactus.obj]



Output : [cactusres.obj]

```
Masukkan file path :
cactus.obj
Masukkan max depth :
5
Masukkan nama file output (.obj) :
cactusres.obj

Banyak voxel yang terbentuk: 502
Banyak vertex yang terbentuk: 4016
Banyak faces yang terbentuk: 6024
Statistik node octree yang terbentuk:
0 : 1
1 : 8
2 : 16
3 : 32
4 : 208
5 : 736
Statistik node yang tidak perlu ditelusuri:
0 : 0
1 : 6
2 : 12
3 : 6
4 : 116
5 : 234
Kedalaman octree: 5
Lama waktu program berjalan: 104.7042 ms
Path file output: cactusres.obj
```


Open
Reset
cactusres.obj

Bab V: Penutup

5.1. Kesimpulan

Setelah melakukan beberapa percobaan, dapat disimpulkan bahwa algoritma *divide and conquer* mampu menyelesaikan permasalahan *voxelization* dengan struktur *octree* secara baik dan terorganisasi. Pendekatan ini membuat proses komputasi menjadi lebih sistematis karena permasalahan utama dipecah menjadi bagian-bagian yang lebih kecil sehingga setiap submasalah dapat ditangani secara lebih sederhana dan efisien. Selain itu, hasil yang diperoleh juga tergolong memuaskan, dengan peningkatan nilai max depth menghasilkan resolusi voxel yang semakin detail dan representatif terhadap model asli. Dengan demikian, penggunaan algoritma *divide and conquer* pada *octree voxelization* dapat dikatakan efektif, baik dari sisi proses maupun kualitas hasil.

5.2. Saran

Penggunaan metode *divide and conquer* pada proses *voxelization* saat ini sudah memberikan hasil yang cukup memuaskan, tetapi masih terdapat ruang untuk pengembangan lebih lanjut. Salah satu peningkatan yang dapat dilakukan adalah mengoptimalkan hasil luaran dengan menghilangkan *vertex* dan *faces* duplikat atau memiliki data yang sama persis sehingga model yang dihasilkan menjadi lebih efisien, baik dari segi penyimpanan maupun pemrosesan. Selain itu, pengembangan lanjutan juga dapat difokuskan pada peningkatan efisiensi algoritma agar waktu eksekusi tetap optimal ketika menangani model dengan kompleksitas yang lebih tinggi. Dengan adanya penyempurnaan tersebut, kualitas hasil *voxelization* diharapkan dapat menjadi lebih rapi, ringan, dan optimal.

Daftar Pustaka

Rinaldi. 2026. *Algoritma Divide and Conquer(Bagian 1)*. Informatika. Diakses pada 18 Maret 2026 Pukul 14.00 WIB dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/07-Algoritma-Divide-and-Conquer-(2026)-Bagian1.pdf

Lampiran

Kode Sumber
https://github.com/alivovue/Tucil2_13524010_13524052.git

Google Drive .obj testing
https://drive.google.com/drive/folders/1psmAqmrY20K4vVltGTYQ7B389sJJLLvN?usp=sharing

Tabel Progress

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan *concurrency*	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj	✓	

Lembar Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Audric Yusuf Maynard Simatupang

Raynard Fausta